BTS Airline Analytics Data Warehouse

1. Executive Summary

The BTS Airline Analytics Data Warehouse is a data engineering project that transforms raw airline operational data into a tested analytics warehouse and Power BI reporting layer. The project uses a modern ELT flow: raw files are stored in an external Backblaze B2 data lake, Snowflake ingests and stores the raw layer, dbt Core performs transformation and testing, and Power BI connects to curated warehouse models for dashboards.

The warehouse is modeled as a Galaxy Schema, also known as a fact constellation, with three fact tables and three shared dimensions. This design supports analysis of flight activity, operational status, delays, cancellations, airlines, airports, and time-based performance.

2. Project Objectives

- Build an end-to-end data engineering pipeline for airline analytics.
- Ingest raw BTS flight files and semi-structured airline and airport metadata.
- Preserve raw data in Snowflake before applying business logic.
- Transform data with dbt into staging models, dimensions, and fact tables.
- Validate data quality using generic tests, singular SQL tests, dbt_utils, and unit tests.
- Deliver Power BI dashboards for operational and analytical decision-making.

3. Data Sources

Source	Format	Usage
BTS TranStats	ZIP to CSV	Flight operations data for 2024 and 2025
OurAirports	JSON	Airport metadata such as airport code, name, type, city, state, country, and coordinates
Skytrax	JSON	Airline metadata such as airline code, airline name, rating, type, hub, and corporate information

The project handles both structured CSV data and semi-structured JSON metadata. Airport and airline JSON files are loaded into Snowflake VARIANT columns before dbt extracts and flattens the required attributes. BTS TranStats is the official source used to select and download the airline on-time performance fields.

4. High-Level Architecture

The pipeline follows a layered ELT architecture.

```
Backblaze B2 External Data Lake
        |
        v
Snowflake RAW Schema
        |
        v
dbt Source and Staging Models
        |
```

```
        v
dbt Mart Layer: Galaxy Schema
        |
        v
Power BI Dashboards
```

Backblaze B2 acts as the external landing zone. Snowflake stores the raw data and provides the warehouse runtime. dbt Core owns the transformation logic, testing, and documentation. Power BI is the final analytics and reporting layer.

5. Snowflake Setup and Ingestion

The Snowflake setup creates the infrastructure required before running dbt. This includes the project database, schemas, virtual warehouse, external stage, raw tables, JSON tables, and ingestion process.

Main resources created:

- Database for the BTS Airline Analytics project.
- RAW schema for landing data with minimal transformation.
- FLIGHT_CORE schema for dbt-generated staging, dimensions, and fact tables.
- Virtual warehouse for compute.
- External stage connected to Backblaze B2.
- Raw flight tables and raw JSON tables.
- Optional teardown and rebuild scripts for development.

Execution order:

Step	Script	Purpose
1	`01_database_schema_setup.sql`	Creates database, schemas, and warehouse
2	`02_storage_integration_backblaze.sql`	Connects Snowflake to Backblaze B2

method |

3	`06_load_raw_json.sql`	Loads airline and airport JSON metadata

The active ingestion design also includes `ingestion_pipeline.py`, an external Python pipeline that streams ZIP files from Backblaze B2 into Snowflake using in-memory processing. It avoids unnecessary local disk usage, filters archive contents, aligns source columns with Snowflake target tables, and writes data efficiently using Snowflake's pandas integration.

6. Python Ingestion Workflow

Two Python scripts were used to automate the movement of raw flight data from the public BTS source into the cloud data lake and then into Snowflake.

6.1 BTS to Backblaze B2 Loader

The first Python script downloads monthly BTS on-time performance ZIP files directly from the BTS PREZIP endpoint:

`https://transtats.bts.gov/PREZIP/On_Time_Reporting_Carrier_On_Time_Performance_1987_present_{year}_{month}.zip`

The script loops through the required year and month values, downloads each ZIP file into memory, and uploads it to Backblaze B2 using the S3-compatible API. Files are stored using Hive-style partitioning:

```

```

```
raw/flights/year=YYYY/month=MM/flights_YYYY_MM.zip
```

This structure makes the data lake organized, readable, and easy to process later by year and month.

Main steps:

- Connect to Backblaze B2 using `boto3`.
- Build the BTS monthly ZIP URL dynamically.
- Download each ZIP file using `requests`.
- Keep the file in RAM using `io.BytesIO`.
- Upload the file to Backblaze B2 with `upload_fileobj`.
- Wait briefly between requests to avoid overloading the source server.

6.2 Backblaze B2 to Snowflake Loader

The second Python script loads the ZIP files from Backblaze B2 into Snowflake raw tables. It connects to the Backblaze bucket, lists all ZIP files under `raw/flights/`, maps each file to the correct target raw table, extracts the CSV file from the ZIP archive in memory, and writes the data into Snowflake.

Target raw tables:

- `BTS_AIRLINE_DB.RAW.RAW_FLIGHTS_2024`
- `BTS_AIRLINE_DB.RAW.RAW_FLIGHTS_2025`
- `BTS_AIRLINE_DB.RAW.RAW_FLIGHTS_2026`

Important implementation details:

- Old raw tables are truncated before loading to keep the dataset clean.
- ZIP files are read into RAM instead of being saved locally.
- The script extracts only the real CSV file and ignores extra archive files.
- Pandas reads all CSV fields as strings to avoid unexpected type conversion during ingestion.
- Column names are normalized and matched dynamically against the Snowflake target table.
- Invalid or accidental columns such as `Unnamed` columns are removed.
- Only columns that already exist in the Snowflake table are loaded.
- `write_pandas` is used for high-performance bulk loading into Snowflake.
- `quote_identifiers=True` protects reserved words such as `YEAR`.

This Python layer acts as the operational bridge between the external data lake and the Snowflake RAW schema. It keeps ingestion automated, repeatable, and separated from dbt transformations.

6.3 Credential Handling

The original scripts contain connection credentials for Backblaze B2 and Snowflake. These credentials must not be included in public documentation or committed to version control. In a production setup, they should be moved to environment variables, a secrets manager, or Snowflake secrets.

7. dbt Transformation Layers

The dbt project separates transformation logic into clear layers.

Layer	Responsibility
Source	Defines raw Snowflake source tables
Stage	Cleans data, casts types, standardizes columns, filters invalid records, and parses JSON

Mart	Builds analytical dimensions and fact tables for the Galaxy Schema

All business logic is handled in dbt instead of the ingestion layer. This keeps raw loading simple and makes transformation logic version-controlled, testable, and reproducible.

All models are materialized into:

`BTS_AIRLINE_DB.FLIGHT_CORE`

8. Warehouse Data Model

The warehouse uses a Galaxy Schema with shared conformed dimensions and multiple related fact tables.

Dimensions:

Table	Description
`dim_date`	Calendar attributes with US Federal Holiday support
`dim_airport`	Airport metadata, used as both origin and destination airport

`dim_airline`	Airline metadata and rating/type information

Fact tables:

Table	Grain
`fact_flight`	Core flight-level data
`fact_flight_operation`	Operational status and timing data for each flight

`fact_flight_delay`	Delay metrics and delay cause breakdown for each flight

Key relationships:

- `Flight_Key` links `fact_flight`, `fact_flight_operation`, and `fact_flight_delay`.
- `Date_Key` links flights to `dim_date`.
- `Airline_Code` links flights to `dim_airline`.
- `Origin_Airport_Code` and `Dest_Airport_Code` both link to `dim_airport`.

9. Schema Details

Core schema fields from the database model:

`fact_flight`

- `Flight_Key`
- `Date_Key`
- `Origin_Airport_Code`
- `Dest_Airport_Code`
- `Airline_Code`
- `Flight_Number`
- `Tail_Number`

- Scheduled_Departure_HHMM
- Scheduled_Arrival_HHMM
- Flight_Distance_Miles
- Actual_Air_Time_Minutes

fact_flight_operation

- Flight_Key
- Actual_Departure_HHMM
- Actual_Arrival_HHMM
- Taxi_Out_Minutes
- Taxi_In_Minutes
- Is_Cancelled
- Cancellation_Reason
- Is_Diverted

fact_flight_delay

- Flight_Key
- Departure_Delay_Minutes
- Arrival_Delay_Minutes
- Is_Departure_Delayed
- Is_Arrival_Delayed
- Carrier_Delay_Minutes
- Weather_Delay_Minutes
- Air_System_Delay_Minutes
- Security_Delay_Minutes
- Late_Aircraft_Delay_Minutes

10. Design Decisions

Why Galaxy Schema:

Flight analytics naturally separates into multiple analytical domains: core flight attributes, operational status, and delay breakdowns. Keeping these as separate fact tables reduces duplication while allowing all facts to share the same date, airport, and airline dimensions.

Why role-playing `dim_airport`:

The same airport dimension is reused twice from `fact_flight`: once for origin airport and once for destination airport. This avoids maintaining duplicate airport tables while still supporting independent departure and arrival airport analysis.

Why raw data is minimally transformed during ingestion:

The ingestion layer is responsible for loading data reliably, not applying business logic. Cleansing, casting, standardization, joins, surrogate key generation, and business rules are handled in dbt where they can be tested and documented.

11. Semi-Structured Data Processing

Airport metadata is stored in Snowflake as JSON and parsed from VARIANT columns. The source is already one row per airport, so nested fields can be accessed directly using Snowflake JSON path expressions without requiring `LATERAL FLATTEN`.

Airline metadata includes nested corporate information. dbt extracts fields such as parent company, headquarters city, and headquarters state from the nested JSON object and exposes them as relational columns.

Examples of parsed metadata:

- Airport code, airport name, airport type, city, state, country, timezone, latitude, longitude, and elevation.
- Airline code, airline name, founded year, airline type, hub airport, airline rating, parent company, and headquarters information.

12. Testing and Data Quality

The project uses a comprehensive dbt testing strategy.

Testing framework:

- dbt generic tests.
- dbt singular SQL tests.
- dbt unit tests.
- `dbt_utils` package.

Unit test coverage:

Model	Unit Tests
`dim_airline`	3
`dim_airport`	4
`dim_date`	4
`fact_flight`	3
`fact_flight_operation`	2
`fact_flight_delay`	2

Total unit tests: 18.

Custom singular test groups:

- Data quality: positive flight distance, valid origin/destination, no future flight dates, valid HHMM time values.
- Business rules: cancelled flights not diverted, cancelled flights have no delay, cancellation reason required, arrival delay requires delay minutes.
- Referential consistency: row count checks across the three fact tables.
- Calendar integrity: day-of-week and holiday consistency checks.

13. Power BI Reporting Layer

Power BI connects to the curated Snowflake warehouse to provide interactive dashboard pages for business analysis.

Recommended dashboard pages:

- Executive overview.
- Flight operations overview.
- Delay analysis.
- Cancellation and diversion analysis.
- Airline performance.
- Airport and route performance.

Core KPIs:

- Total flights.
- Average departure delay.

- Average arrival delay.
- On-time rate.
- Cancellation rate.
- Diversion rate.
- Delay cause breakdown.
- Taxi time and air time analysis.

14. Security and Operational Notes

- Never expose Backblaze B2 keys, Snowflake usernames, or Snowflake passwords inside project documentation, screenshots, GitHub repositories, or shared notebooks.
- Store sensitive values in environment variables or a secure secrets manager.
- Keep raw ingestion scripts parameterized so the same logic can run safely in development and production.
- Run Snowflake setup and ingestion before executing `dbt build`.
- Keep the RAW layer unchanged and apply all analytical business logic in dbt.

15. Deliverables

Project deliverables include:

- Snowflake setup scripts.
- External storage integration with Backblaze B2.
- Raw flight and metadata ingestion process.
- dbt transformation project.
- Galaxy Schema warehouse models.
- dbt tests and documentation.
- Power BI dashboard file/pages.
- Final presentation.
- Final technical documentation.

16. Conclusion

The BTS Airline Analytics Data Warehouse provides a complete data engineering workflow from external raw storage to governed analytics. By combining Backblaze B2, Snowflake, dbt Core, and Power BI, the project creates a reproducible and testable pipeline for airline performance reporting.

The Galaxy Schema design supports flexible analysis across flight operations, delays, cancellations, airlines, airports, and dates. The testing layer improves reliability and gives confidence that the dashboard insights are based on validated transformation logic.